

فاز دوم

کیوان ثاقب مجیدزاده

موضوع برنامه

هدف این برنامه بررسی یک فرمول یا یک رابطه استنتاج در منطق گزاره‌ای است. برنامه صفر یا چند مقدمه و یک نتیجه دریافت می‌کند و بر اساس تعداد مقدمات یکی از دو کار زیر را انجام می‌دهد:

۱. اگر تعداد مقدمات صفر باشد، فرمول نتیجه را در یکی از سه وضعیت Valid، Satisfiable یا Unsatisfiable قرار می‌دهد.

۲. اگر حداقل یک مقدمه وجود داشته باشد، برقرار بودن استنتاج را بررسی می‌کند. خروجی در این حالت Valid یا Invalid است و برای حالت Invalid، اولین مثال نقض نیز چاپ می‌شود.

برنامه برای محاسبه هم‌زمان مقدار فرمول در تمام valuationها از یک عدد صحیح به عنوان bitset استفاده می‌کند. هر بیت از این عدد، نتیجه فرمول در یک valuation را نشان می‌دهد. به این ترتیب، به جای ارزیابی جداگانه فرمول برای هر حالت، بسیاری از محاسبات با عملگرهای بیتی روی اعداد صحیح انجام می‌شوند.

تعریف وضعیت‌های خروجی

اگر هیچ مقدمه‌ای وجود نداشته باشد، وضعیت فرمول نتیجه به صورت زیر تعیین می‌شود:

- Valid: فرمول در تمام valuationها درست است.
- Satisfiable: فرمول در حداقل یک valuation درست است، اما در همه آن‌ها درست نیست.
- Unsatisfiable: فرمول در هیچ valuation درست نیست.

اگر یک یا چند مقدمه وجود داشته باشد، استنتاج زیر بررسی می‌شود:

$$\varphi_1, \varphi_2, \dots, \varphi_n \models \psi.$$

این استنتاج زمانی برقرار است که هر valuation که همه مقدمات را درست می‌کند، نتیجه ψ را نیز درست کند. اگر valuationای وجود داشته باشد که تمام مقدمات را درست و نتیجه را نادرست کند، خروجی Invalid است و آن valuation یک مثال نقض محسوب می‌شود.

اگر هیچ valuationی نتواند تمام مقدمات را هم‌زمان درست کند، استنتاج به صورت vacuous برقرار است و برنامه Valid چاپ می‌کند.

نمادهای قابل قبول

نماد متنی	معنی
a تا z	متغیر گزاره‌ای تک حرفی
T	ثابت درست
F	ثابت نادرست
$\neg A$	نقیض A
$(A \& B)$	عطف $A \wedge B$
$(A B)$	فصل $A \vee B$
$(A \rightarrow B)$	شرطی $A \rightarrow B$
$(A \leftrightarrow B)$	دوشرطی $A \leftrightarrow B$

متغیرهای گزاره‌ای فقط حروف کوچک انگلیسی هستند. هر زیرفرمول دوتایی باید داخل یک جفت پرانتز قرار گیرد. نقیض پرانتز جداگانه نمی‌گیرد و مستقیماً پیش از زیرفرمول بعدی نوشته می‌شود. فرمول ورودی نیز نباید فاصله اضافی داشته باشد.

قالب ورودی

خط اول ورودی عدد n ، یعنی تعداد مقدمات، است:

$$0 \leq n \leq 20.$$

پس از آن:

- اگر $n = 0$ باشد، خط بعدی فقط شامل فرمول نتیجه است.
- اگر $n \geq 1$ باشد، ابتدا n خط شامل مقدمات و سپس یک خط شامل فرمول نتیجه خوانده می‌شود.

برای نمونه، ورودی زیر شامل دو مقدمه و یک نتیجه است:

2
(p→q)
p
q

قالب خروجی

برای $n = 0$ تنها یکی از سه برچسب زیر چاپ می‌شود:

Valid
Satisfiable
Unsatisfiable

برای $n \geq 1$ ، اگر استنتاج برقرار باشد، خروجی فقط Valid است. اگر استنتاج برقرار نباشد، خط اول Invalid و خط دوم اولین مثال نقض است:

Invalid
p=False q=True

تمام متغیرها در خط دوم به ترتیب الفبایی چاپ می‌شوند. برای هر متغیر، False پیش از True قرار می‌گیرد.

ایده اصلی استفاده از bitset

فرض می‌کنیم k متغیر متمایز در ورودی وجود دارد. تعداد تمام valuation‌های ممکن برابر است با:

$$V = 2^k.$$

برنامه برای نتیجه هر فرمول یک عدد صحیح با V بیت می‌سازد. بیت شماره i ، نتیجه آن فرمول را در valuation شماره i نشان می‌دهد. مقدار هر بیت مطابق جدول زیر تعیین می‌شود:

bit _{i}	وضعیت فرمول در valuation شماره i
1	درست
0	نادرست

متغیر full_mask عددی است که تمام V بیت آن برابر یک هستند:

$$\text{full_mask} = 2^V - 1.$$

این مقدار نتیجه ثابت T در تمام valuation‌ها را نمایش می‌دهد. عدد صفر نیز نتیجه ثابت F است.

ترتیب valuation‌ها

متغیرها ابتدا به ترتیب الفبایی مرتب می‌شوند. سپس برای هر متغیر، False پیش از True در نظر گرفته می‌شود. برای دو متغیر p و q ترتیب به صورت زیر است:

شماره بیت	p	q
۰	False	False
۱	False	True
۲	True	False
۳	True	True

بنابراین bit mask متغیر p از بیت کم‌ارزش به پرارزش برابر 0011 و bit mask متغیر q برابر 0101 است. هنگام نمایش متنی معمول عدد دودویی، بیت پرارزش در سمت چپ چاپ می‌شود؛ در نتیجه رشته‌های ساخته شده در کد برای p و q به ترتیب 1100 و 1010 هستند.

ساختار کلی اجرای برنامه

مراحل اجرای برنامه به ترتیب زیر است:

۱. خواندن تعداد مقدمات، خود مقدمات و نتیجه در تابع main؛

۲. استخراج همه متغیرهای گزاره‌ای و مرتب کردن آنها؛
۳. ساخت bit mask هر متغیر و مقدار full_mask؛
۴. تجزیه و ارزیابی هر فرمول با کلاس FormulaParser؛
۵. بررسی وضعیت فرمول نتیجه برای $n = 0$ ؛
۶. ترکیب نتایج مقدمات برای $n \geq 1$ ؛
۷. یافتن اولین مثال نقض در صورت برقرار نبودن استنتاج؛
۸. چاپ برجسب خروجی و در صورت نیاز، مقادیر متغیرهای مثال نقض.

کلاس FormulaParser

این کلاس فرمول را از چپ به راست تجزیه می‌کند، اما برخلاف برنامه فاز اول، برای فرمول یک درخت جداگانه نمی‌سازد. هر فراخوانی parse_formula مستقیماً یک عدد صحیح bitset برمی‌گرداند که نتیجه زیرفرمول فعلی در تمام valuation‌ها است.

ویژگی‌های این کلاس عبارت‌اند از:

ویژگی	توضیح
text	متن فرمول ورودی
variable_masks	نگاشت هر متغیر به bit mask آن
full_mask	عددی که تمام بیت‌های معتبر آن یک هستند
position	محل فعلی خواندن متن فرمول

متد parse، متد parse_formula را فراخوانی می‌کند و سپس بررسی می‌کند که تمام متن ورودی مصرف شده باشد. اگر نویسه‌ای اضافه باقی مانده باشد، خطای ValueError ایجاد می‌شود.

تجزیه و ارزیابی زیرفرمول‌ها

متد parse_formula حالت‌های زیر را پردازش می‌کند:

- متغیر: bit mask ذخیره شده برای آن متغیر برگردانده می‌شود.
- ثابت T: مقدار full_mask برگردانده می‌شود.
- ثابت F: مقدار صفر برگردانده می‌شود.
- نقیض: bitset زیرفرمول بعدی محاسبه و تمام بیت‌های معتبر آن معکوس می‌شوند.
- زیرفرمول دوتایی: طرف چپ، عملگر و طرف راست خوانده می‌شوند و نتیجه با متد apply_operator محاسبه می‌شود.

برای نقیض نمی‌توان مستقیماً از عملگر ~ پایتون استفاده کرد؛ زیرا این عملگر تمام بیت‌های نامحدود عدد صحیح پایتون را معکوس می‌کند. کد برای محدود کردن نقیض به بیت‌های معتبر از رابطه زیر استفاده می‌کند:

$$\neg A = \text{full_mask} \wedge A.$$

پردازش عملگرهای دوتایی

متد `parse_operator` ابتدا عملگرهای بلندتر را بررسی می‌کند. بنابراین <-> پیش از >- و سپس & و | آزمایش می‌شوند.

متد `apply_operator` عملگرها را به صورت زیر روی `bitset`ها اجرا می‌کند:

عملیات بیتی	رابطه منطقی	عملگر
<code>left & right</code>	$A \wedge B$	<code>&</code>
<code>left right</code>	$A \vee B$	<code> </code>
<code>(full_mask ^ left) right</code>	$\neg A \vee B$	<code>-></code>
<code>full_mask ^ (left ^ right)</code>	$\neg(A \oplus B)$	<code><-></code>

در عملگر دوطرفه، هر بیت زمانی یک می‌شود که بیت‌های متناظر طرف چپ و راست یکسان باشند. ابتدا XOR اختلاف دو طرف را مشخص می‌کند و سپس نتیجه در محدوده `full_mask` معکوس می‌شود.

تابع collect_variables

این تابع تمام فرمول‌ها، یعنی مقدمات و نتیجه، را بررسی می‌کند. هر حرف کوچک انگلیسی در یک set قرار می‌گیرد تا تکرارها حذف شوند. در پایان، متغیرها با تابع sorted به ترتیب الفبایی برگردانده می‌شوند. مرتب‌بودن این فهرست ضروری است، زیرا ترتیب valuationها و ترتیب چاپ متغیرها در مثال نقض به همین ترتیب وابسته است.

تابع build_variable_masks

این تابع برای هر متغیر یک bit mask می‌سازد و همراه آن full_mask را برمی‌گرداند. اگر تعداد متغیرها k باشد:

$$\text{valuation_count} = 2^k,$$

$$\text{full_mask} = 2^{\text{valuation_count}} - 1.$$

برای هر متغیر، اندازه بلوک‌های متوالی False و True از رابطه زیر به دست می‌آید:

$$\text{block_size} = 2^{k-i-1},$$

که در آن i محل متغیر در فهرست مرتب‌شده است. سپس الگویی شامل یک بلوک یک‌ها و یک بلوک صفرها ساخته و به تعداد لازم تکرار می‌شود:

```
pattern = ("1" * block_size) + ("0" * block_size)
bit_string = pattern * repetitions
```

قرارگرفتن بلوک یک‌ها پیش از صفرها در رشته با نحوه نمایش اعداد دودویی هماهنگ است؛ بیت شماره صفر در سمت راست رشته قرار دارد. در نتیجه، valuation شماره صفر همچنان حالتی است که تمام متغیرها False هستند.

تابع evaluate_formula

این تابع یک نمونه از FormulaParser می‌سازد و متد parse را فراخوانی می‌کند. خروجی آن یک عدد صحیح است که نتیجه فرمول را در تمام valuationها نگه می‌دارد.

جدا کردن این تابع از کلاس باعث شده است بخش solve بدون نیاز به اطلاع از جزئیات تجزیه، هر مقدمه یا نتیجه را ارزیابی کند.

تابع solve و پردازش ورودی

تابع solve ابتدا خط‌های ورودی را جدا و فاصله‌های ابتدا و انتهای آن‌ها را حذف می‌کند. خط‌های خالی انتهایی کنار گذاشته می‌شوند. سپس تعداد مقدمات از خط اول خوانده می‌شود.

اگر تعداد مقدمات صفر باشد، فهرست مقدمات خالی است و خط بعدی به عنوان نتیجه در نظر گرفته می شود. در غیر این صورت، n خط بعدی مقدمات و خط پس از آن نتیجه است. پس از استخراج متغیرها و ساخت bit mask، نتیجه با تابع `evaluate_formula` ارزیابی می شود.

بررسی حالت بدون مقدمه

اگر $n = 0$ باشد، که نتیجه فرمول را با صفر و `full_mask` مقایسه می کند:

خروجی	شرط
Valid	<code>conclusion_result == full_mask</code>
Unsatisfiable	<code>conclusion_result == 0</code>
Satisfiable	هر مقدار دیگر

برای مثال، فرمول $p \vee \neg p$ یک tautology است؛ بنابراین تمام بیت های نتیجه یک هستند و خروجی Valid خواهد بود.

ترکیب مقدمات

اگر حداقل یک مقدمه وجود داشته باشد، برنامه باید valuation هایی را پیدا کند که تمام مقدمات در آن ها درست هستند. به همین دلیل `premises_result` ابتدا برابر `full_mask` قرار می گیرد و سپس نتیجه هر مقدمه با عملگر بیتی AND به آن اضافه می شود:

```
premises_result &= premise_result
```

پس از پردازش تمام مقدمات، هر بیت یک در `premises_result` نشان می دهد که همه مقدمات در valuation متناظر درست هستند.

اگر در میان محاسبات مقدار `premises_result` صفر شود، هیچ valuation وجود ندارد که تمام مقدمات را درست کند. در این حالت استنتاج به صورت vacuous برقرار است و تابع بدون ادامه پردازش، Valid را برمی گرداند.

ساخت ماسک مثال های نقض

یک مثال نقض باید تمام مقدمات را درست و نتیجه را نادرست کند. ماسک valuation هایی که نتیجه در آن ها نادرست است از رابطه زیر به دست می آید:

```
full_mask ^ conclusion_result.
```

سپس برنامه آن را با ماسک مقدمات عطف می کند:

```
counterexample_mask = premises_result & (full_mask ^ conclusion_result).
```

اگر counterexample_mask صفر باشد، مثال نقضی وجود ندارد و استنتاج Valid است. در غیر این صورت، هر بیت یک در این ماسک یک مثال نقض را نشان می‌دهد.

یافتن اولین مثال نقض

به دلیل ترتیب ساخت valuationها، کوچک‌ترین شمارهٔ بیت یک در counterexample_mask همان اولین مثال نقض مورد انتظار است.

تابع find_first_set_bit ابتدا فقط کم‌ارزش‌ترین بیت یک را جدا می‌کند:

```
lowest_set_bit = x & (-x).
```

سپس شمارهٔ آن بیت با رابطهٔ زیر به دست می‌آید:

```
index = lowest_set_bit.bit_length() - 1.
```

این روش بدون پیمایش تک‌تک بیت‌ها، شمارهٔ اولین valuation مناسب را پیدا می‌کند.

تابع format_valuation

این تابع شمارهٔ valuation و فهرست مرتب‌شدهٔ متغیرها را دریافت می‌کند. برای متغیر قرارگرفته در موقعیت i ، بیت مناسب با رابطهٔ زیر خوانده می‌شود:

```
value = (valuation_index >> (k - i - 1)) & 1.
```

مقدار صفر به False و مقدار یک به True تبدیل می‌شود. سپس رشته‌هایی مانند p=False و q=True با فاصله به هم متصل می‌شوند.

اگر هیچ متغیری در ورودی وجود نداشته باشد، فهرست نتیجه خالی است و این تابع یک رشتهٔ خالی برمی‌گرداند. بنابراین در حالت Invalid، خط دوم خروجی به‌درستی خالی چاپ می‌شود.

تابع main

تابع main تعداد مقدمات را با input می‌خواند. سپس دقیقاً به همان تعداد فرمول مقدمه و در پایان یک فرمول نتیجه دریافت می‌کند:

```
premise_count = int(input().strip())
premises = [
    input().strip()
```

```
    for _ in range(premise_count)
]
conclusion = input().strip()
```

خط‌های خوانده‌شده دوباره با `join` به `"\n"` به یک رشته تبدیل و به تابع `solve` ارسال می‌شوند. در پایان، مقدار برگشتی `solve` چاپ می‌شود.

اجرای نمونه اول: فرمول Valid

ورودی:

0
(p|!p)

برای متغیر p دو valuation وجود دارد. فرمول $p \vee \neg p$ در هر دو حالت درست است؛ بنابراین bitset نتیجه با full_mask برابر می شود.

خروجی:

Valid

اجرای نمونه دوم: استنتاج Invalid

ورودی:

1
(p|q)
p

در valuation برابر با p=False q=True، مقدمه $p \vee q$ درست است، اما نتیجه p نادرست است. این حالت اولین مثال نقض بر اساس ترتیب تعیین شده است.

خروجی:

Invalid
p=False q=True

اجرای نمونه سوم: استنتاج vacuous

ورودی:

2
p
!p
q

هیچ valuationی نمی تواند هر دو مقدمه p و $\neg p$ را همزمان درست کند. بنابراین مقدار premises_result صفر می شود و استنتاج به صورت vacuous برقرار است.

خروجی:

Valid

پیچیدگی زمانی و حافظه

اگر تعداد متغیرهای متمایز را k و تعداد valuation ها را $V = 2^k$ در نظر بگیریم، اندازه هر bitset برابر V بیت است.

ساخت ماسک متغیرها در مجموع به فضایی متناسب با kV بیت نیاز دارد. عملگرهای بیتی پایتون نیز باید اعداد صحیح V -بیتی را پردازش کنند؛ بنابراین هزینه آنها با تعداد بیت‌ها افزایش می‌یابد.

اگر مجموع طول تمام فرمول‌ها را L و تعداد فرمول‌ها را m در نظر بگیریم، بخش تجزیه هر نویسه را یک‌بار بررسی می‌کند، اما عملیات هر گره روی یک bitset با اندازه V انجام می‌شود. در نتیجه، عامل غالب با رشد تعداد متغیرها نمایانی است:

$$V = 2^k.$$

این روش برای حداکثر ۲۰ متغیر قابل استفاده است، زیرا تمام valuation ها به صورت فشرده در بیت‌های اعداد صحیح پایتون ذخیره می‌شوند و عملیات روی بسیاری از آنها به صورت هم‌زمان انجام می‌شود.

محدودیت‌های برنامه

- نام هر متغیر فقط یک حرف کوچک انگلیسی از a تا z است.
- زیرفرمول‌های دوتایی باید به‌طور کامل داخل پرانتز قرار گیرند.
- فرمول‌ها نباید فاصله اضافی داشته باشند.
- تعداد valuation ها با تعداد متغیرها به صورت نمایانی رشد می‌کند.
- تابع solve معتبر بودن تعداد خط‌های ورودی را به‌طور مستقل بررسی نمی‌کند و به قالب تضمین‌شده ورودی متکی است.
- افزایش حد بازگشت به 10000 امکان پردازش فرمول‌های عمیق‌تر را فراهم می‌کند، اما ورودی بسیار عمیق همچنان می‌تواند منابع زیادی مصرف کند.